Knowledge graphs, ontologías y capa semántica en sistemas de IA agéntica

Material de lectura — Arquitectura de Agentes IA (posgrado) · Versión de trabajo, 2026-10-01

Resumen y objetivos de aprendizaje

Los sistemas de IA agéntica empresariales suelen fallar menos por falta de capacidad del modelo que por falta de *significado compartido*: el agente no sabe qué quiere decir «cliente activo» en cada sistema, cómo se relacionan las entidades del negocio, ni cuál de todos los hechos disponibles importa para la decisión que debe tomar. Este reporte sintetiza una colección de materiales de divulgación técnica sobre ese problema y los ordena de lo básico a lo avanzado.

Al terminar la lectura, el alumno debería poder:

1. Distinguir con precisión *taxonomía*, *ontología*, *modelo semántico*, *capa semántica*, *mapeo semántico* y *knowledge graph*, y explicar qué pregunta responde cada uno.
2. Describir la arquitectura de un knowledge graph empresarial, desde la extracción de entidades hasta su uso en GraphRAG y agentes.
3. Explicar por qué un grafo de conocimiento no basta para que un agente decida bien, y qué aportan los grafos de contexto y los grafos procedimentales.
4. Evaluar críticamente cuándo conviene (y cuándo no) pagar el costo de un grafo frente a una búsqueda vectorial.
5. Reconocer los límites de la proveniencia como mecanismo de auditoría de agentes.

1. Motivación: de la similitud al significado

La recuperación vectorial responde a una sola pregunta: «¿qué contenido se parece a esta consulta?». Muchas preguntas empresariales, en cambio, piden otra cosa: «¿cómo se conectan estas entidades, en qué momento y según qué evidencia?». Un *embedding* captura similitud semántica; un grafo captura relaciones explícitas, jerarquías, tiempo y trazabilidad. La tesis que atraviesa el material es que la IA empresarial necesita ambos, y que los grafos solo rinden cuando descansan sobre una definición previa y gobernada del significado de los datos.

2. Vocabulario: seis términos que se confunden

El primer obstáculo es terminológico. Los términos *modelo semántico, ontología* y *knowledge graph* se usan como sinónimos en muchas conversaciones de arquitectura, y no lo son. La Figura 1 resume la distinción en tres preguntas.

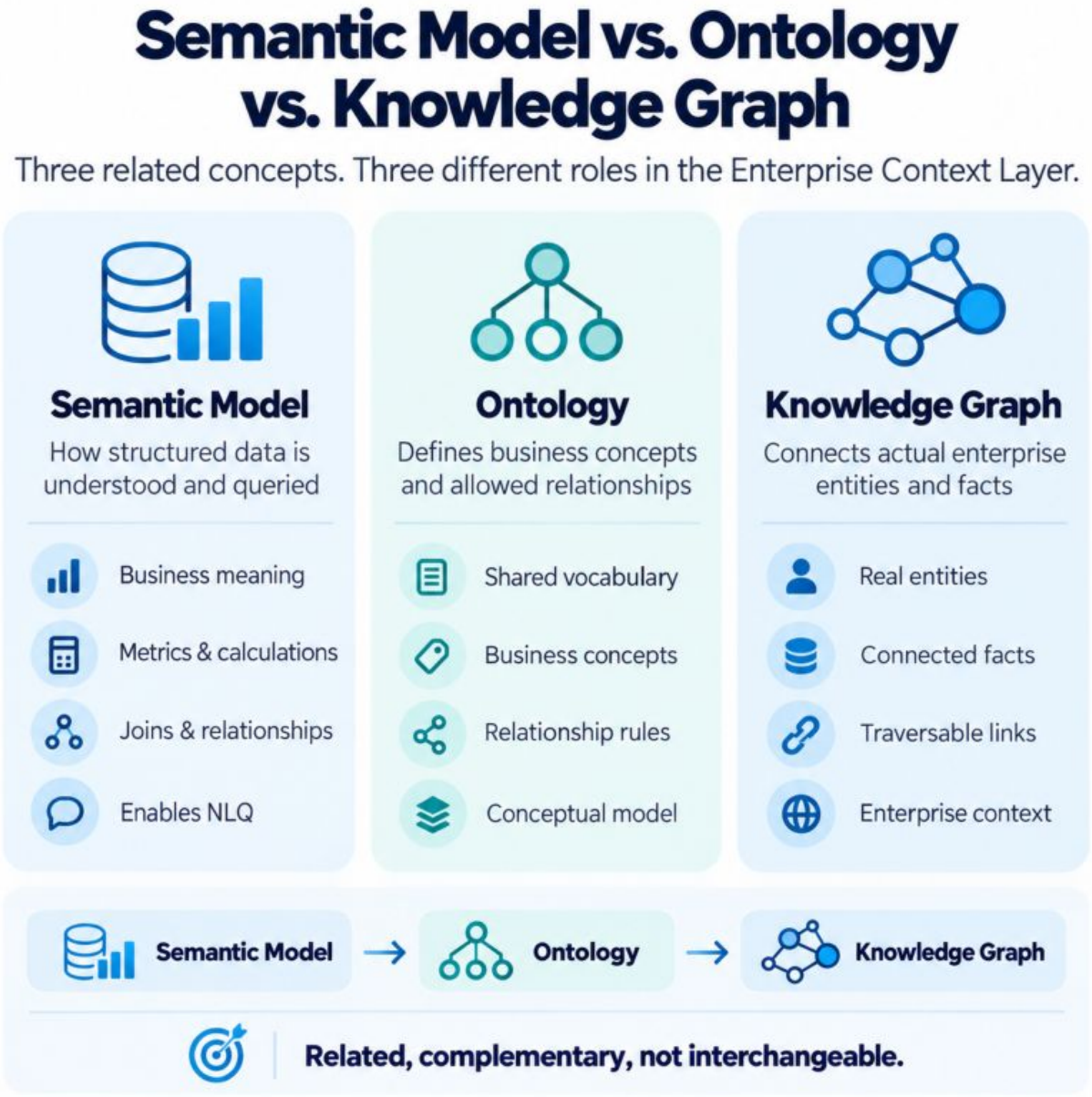


Figura 1. Modelo semántico, ontología y knowledge graph como tres capas complementarias de contexto empresarial. Fuente: material de divulgación (copia local en fuentes/ 7508657406294298624/).

- El **modelo semántico** responde «¿qué significan los datos estructurados y cómo se consultan?». Define nombres de negocio, métricas y cálculos, dimensiones y las relaciones (*joins*) entre tablas. Es lo que permite resolver, por ejemplo, la consulta en lenguaje natural «cuentas por cobrar abiertas por cliente en la región Noreste en los últimos 90 días»: el sistema sabe qué es «abierto», qué tablas usar y qué campo de fecha manda.
- La **ontología** responde «¿qué conceptos existen en el negocio y cómo pueden relacionarse?». Es un modelo conceptual: define clases, jerarquías, restricciones y tipos de relación (un cliente *realiza* pedidos, un pedido *genera* facturas), sin referirse a una tabla concreta.

- El **knowledge graph** responde «¿cuáles son las entidades y relaciones reales?». Aplica los conceptos de la ontología a hechos concretos: *Acme Corporation* → *realizó* → *Pedido 45873* → *generó* → *Factura 90231*.

La Figura 2 amplía el panorama con otros dos términos, la **taxonomía** (clasificación jerárquica, solo relaciones «es-un») y el **mapeo semántico** (correspondencias entre vocabularios de sistemas distintos, p. ej. «Customer» en el CRM equivale a «Client» en el ERP).



Figura 2. Taxonomía, ontología, mapeo semántico y knowledge graph: diferencias, flujo de madurez y errores frecuentes. Fuente: material de divulgación (copia local en fuentes/7478312221530312705/).

La figura propone un «flujo de madurez» (taxonomía → ontología → mapeo semántico → knowledge graph) con expresividad, complejidad y valor de negocio crecientes, y lista errores típicos: construir un grafo sin ontología clara (resulta un grafo desordenado e

inconsistente), usar una taxonomía cuando se necesita semántica rica, mapear todo sin modelar nada, sobremodelar demasiado pronto, e ignorar la gobernanza.

Tabla 1. Síntesis comparativa (elaboración propia a partir del material).

Concepto	Pregunta que responde	Relaciones	Ejemplo
Taxonomía	¿A qué categoría pertenece?	Solo jerárquicas (es-un)	Producto > Electrónica > Laptop
Ontología	¿Qué significa y cómo se relaciona?	Muchos tipos, con restricciones	Cliente realiza Pedido
Mapeo semántico	¿Cómo se corresponde el sistema A con el B?	Equivalencias entre modelos	CRM «Customer» ↔ ERP «Client»
Modelo semántico	¿Cómo se entienden y consultan los datos estructurados?	Joins, métricas, dimensiones	«Cuentas por cobrar abiertas»
Knowledge graph	¿Qué entidades existen y cómo se conectan?	Entidades reales conectadas	Acme → realizó → Pedido 45873

Una precisión conceptual. El material presenta la secuencia anterior como una jerarquía didáctica, pero conviene no tomarla al pie de la letra. En los estándares de la web semántica (RDF, RDFS, OWL) una ontología puede expresarse ella misma como un grafo, y se suele distinguir entre el nivel de esquema (conceptos y reglas, *TBox*) y el de instancias (hechos, *ABox*). Desde esa óptica, ontología y grafo no son «capas» apiladas sino dos aspectos de una misma representación. *(Nota: esta precisión proviene de conocimiento estándar del área, no del material de origen; debe contrastarse con la documentación de W3C antes de la versión final.)*

3. Los seis conceptos de la IA empresarial, visualizados

La Figura 3 reúne seis conceptos que aparecen juntos en casi cualquier arquitectura de IA empresarial, con un ejemplo gráfico de cada uno.

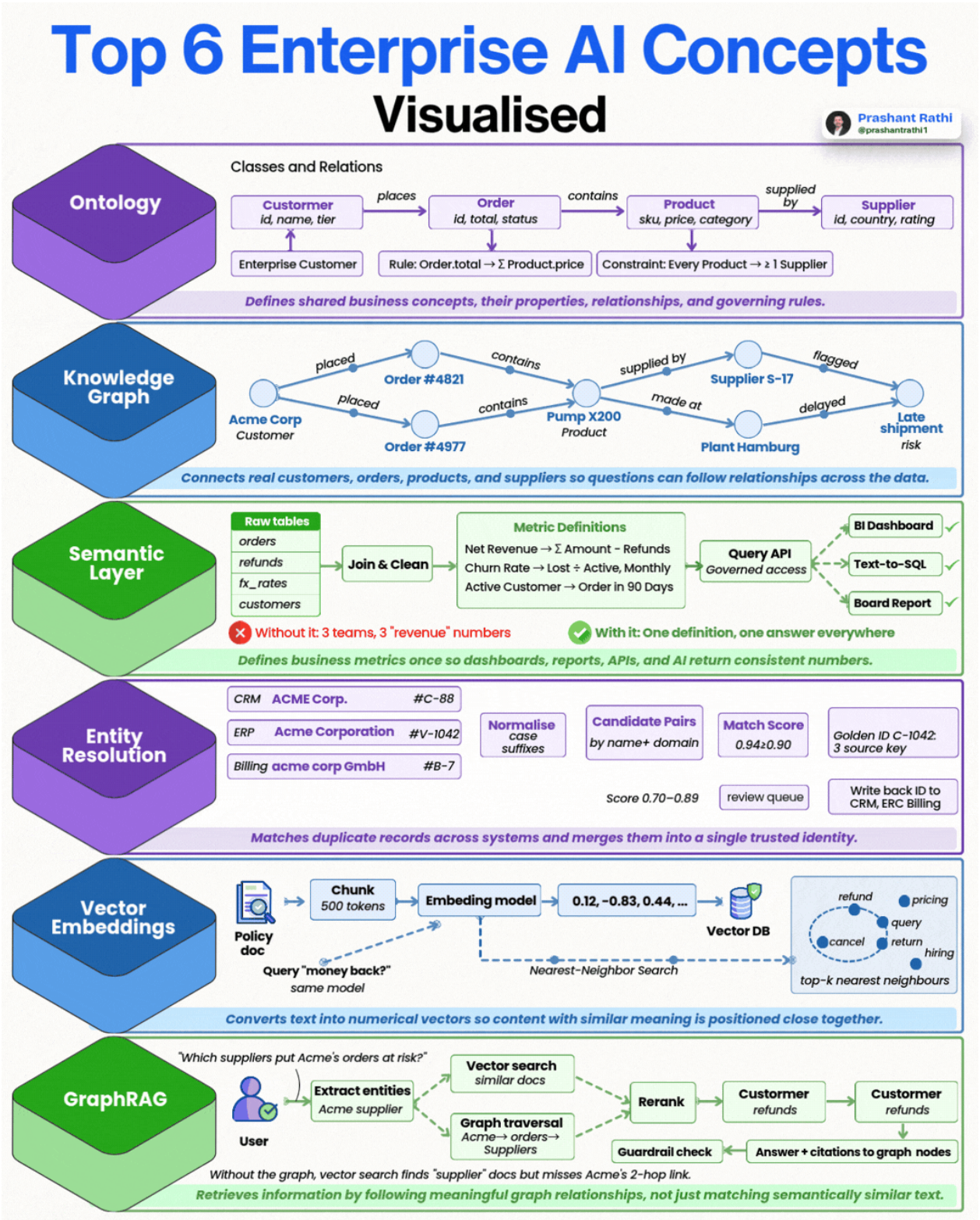


Figura 3. Ontología, knowledge graph, capa semántica, resolución de entidades, embeddings vectoriales y GraphRAG. Fuente: material de divulgación (copia local en fuentes/ 7511047409553199104/).

Cada concepto se puede fijar con una palabra clave:

Concepto	Idea clave	Qué aporta
Ontología	Significado	Define qué es un Cliente, un Pedido, un Proveedor y cómo se conectan, con reglas y restricciones
Knowledge graph	Conexiones	Conecta entidades reales para que las preguntas puedan seguir relaciones a través de los datos
Capa semántica	Consistencia	Define cada métrica una vez, de modo que tableros, APIs y herramientas de IA devuelvan los mismos números
Resolución de entidades	Identidad	Detecta que «ACME Corp.», «Acme Corporation» y «acme corp GmbH» son la misma empresa y los une bajo un identificador confiable
Embeddings vectoriales	Similitud	Representan texto como vectores para hallar contenido de significado cercano
GraphRAG	Recuperación con contexto	Combina búsqueda semántica con el recorrido de relaciones entre entidades

El ejemplo de la figura para GraphRAG es didáctico: ante «¿qué proveedores ponen en riesgo los pedidos de Acme?», una búsqueda puramente vectorial encuentra documentos sobre «proveedores» pero puede no llegar al vínculo de dos saltos entre Acme, sus pedidos y un proveedor marcado como riesgoso; recorrer el grafo sí lo hace.

Un glosario ampliado

Para quien necesite el vocabulario completo del área, la Figura 4 ofrece una tabla de 22 términos con su definición, su razón de ser y un uso empresarial típico.

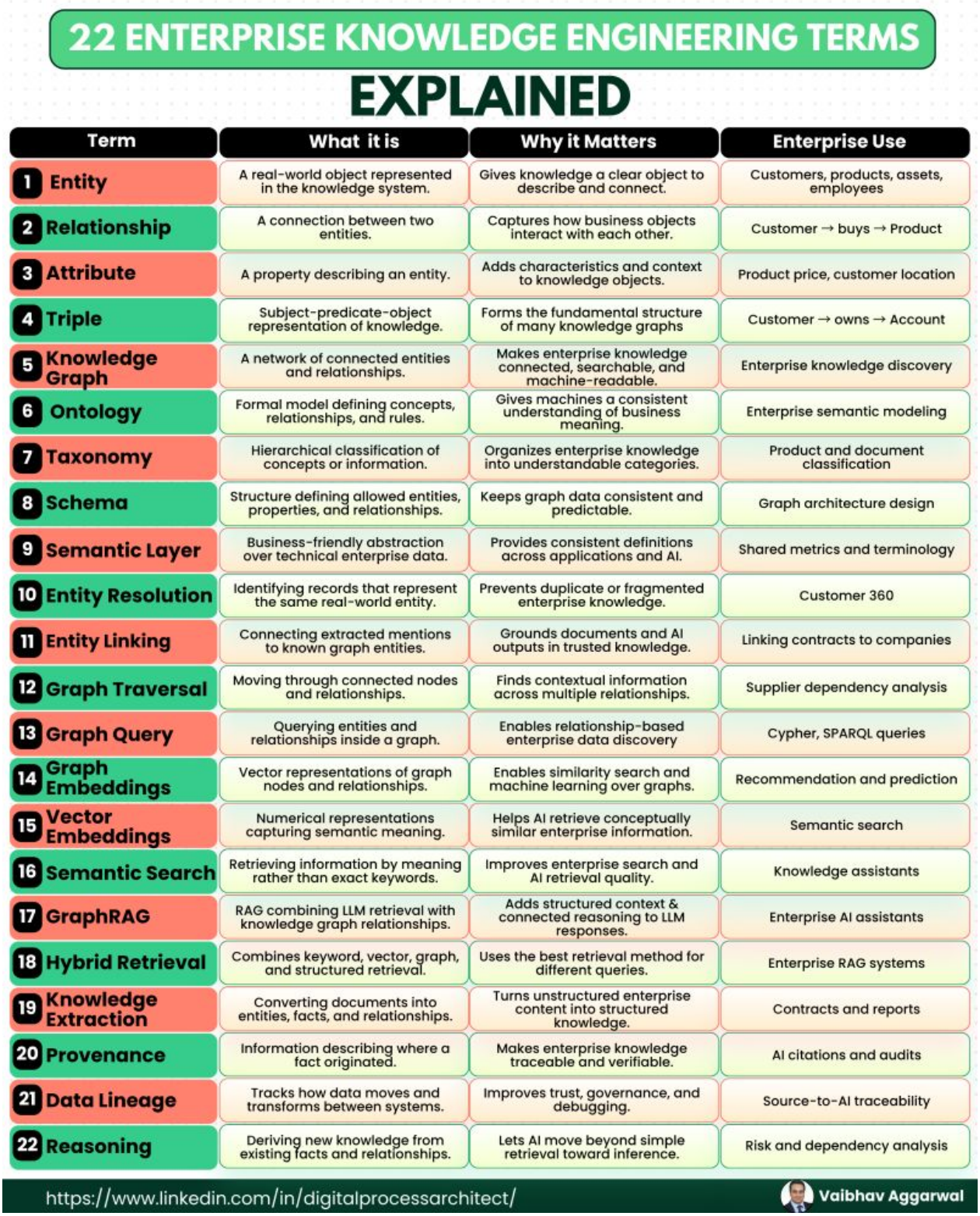


Figura 4. Veintidós términos de ingeniería del conocimiento empresarial, de «entidad» a «razonamiento». Fuente: material de divulgación (copia local en fuentes/7507426768228679680/).

Tres grupos de términos conviene distinguir: los **elementos del grafo** (entidad, relación, atributo, tripleta, esquema), las **operaciones** (enlace de entidades, resolución, extracción de conocimiento, recorrido y consulta del grafo) y las **garantías**

(proveniencia y linaje de datos, que hacen trazable y verificable el conocimiento). Los lenguajes de consulta mencionados son Cypher y SPARQL.

4. La capa semántica como cimiento

Una de las tesis más repetidas del material es de orden práctico: *definir el significado antes de las relaciones*. Muchas organizaciones construyen primero el grafo o el agente y dejan para después el acuerdo sobre qué significa cada concepto. El resultado son definiciones inconsistentes, productos de datos difíciles de reutilizar y agentes que no logran razonar con contexto empresarial confiable.

La **capa semántica** se describe como una capa de metadatos y abstracción de negocio *encima* de los productos de datos; no almacena datos de negocio. Aporta vocabulario, definiciones estándar, entidades canónicas, mapeos a productos de datos, metadatos de gobierno, relaciones de negocio y responsables o políticas. La Figura 5 muestra la cadena de dependencias propuesta y un proceso de nueve pasos para construirla, ilustrado con un caso bancario.

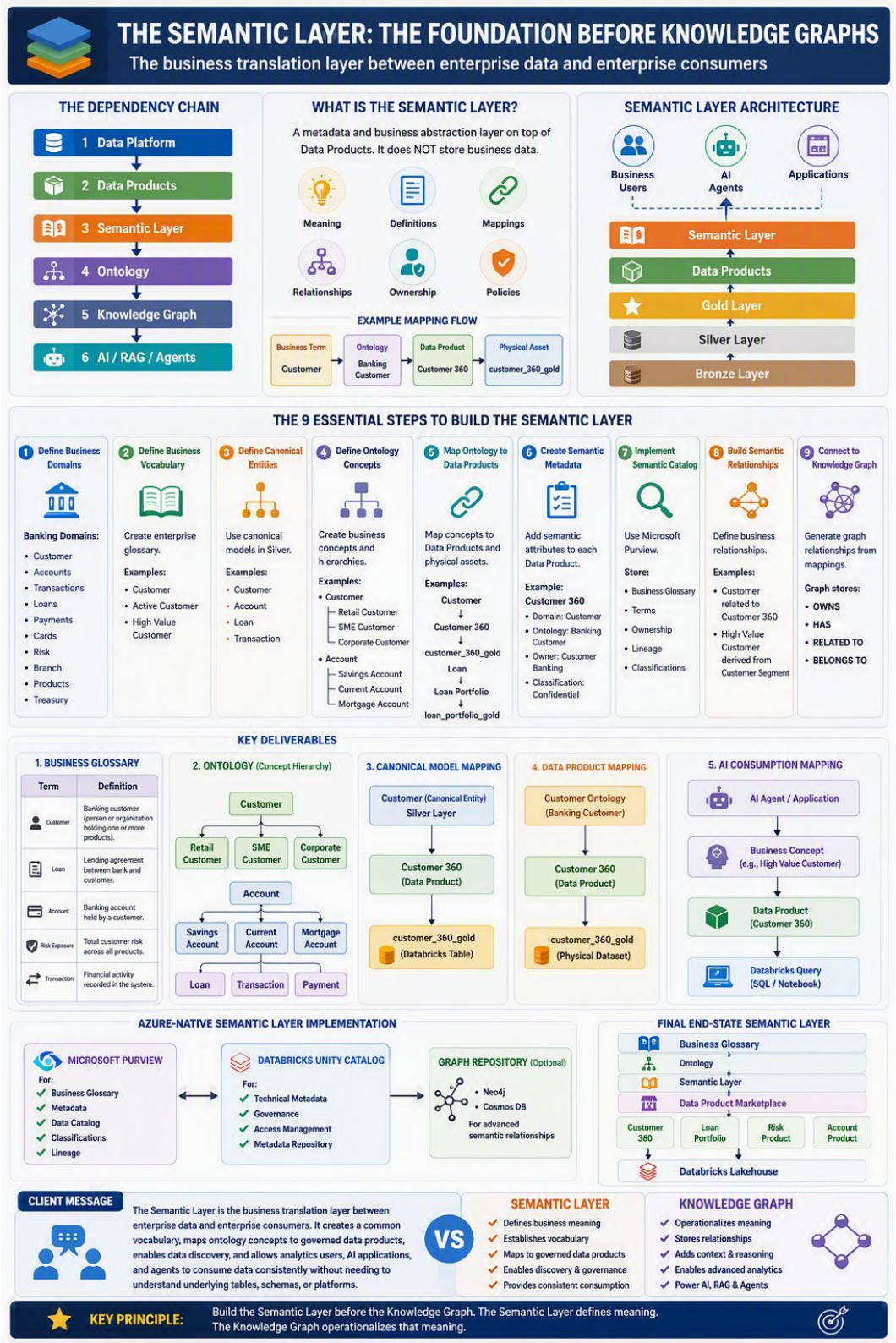


Figura 5. La capa semántica como fundamento previo al knowledge graph: cadena de dependencias, pasos de construcción y entregables. Fuente: material de divulgación (copia local en fuentes/7508064074430181376/).

La cadena es: plataforma de datos → productos de datos → capa semántica → ontología → knowledge graph → IA/RAG/agentes. En el material, la capa semántica *define* el significado, el grafo lo *conecta* y los agentes lo *consumen*. Los pasos de construcción

van de definir dominios y vocabulario de negocio, pasando por entidades canónicas, conceptos ontológicos y su mapeo a productos de datos, hasta añadir metadatos, catálogo, relaciones semánticas y, opcionalmente, un repositorio de grafos.

Dos advertencias para el lector crítico:

- La implementación que muestra la parte inferior de la figura (Microsoft Purview, Databricks Unity Catalog, Neo4j o Cosmos DB) es **una elección tecnológica concreta**, no parte del concepto. La arquitectura es transferible a otras plataformas.
- La pregunta que el material deja abierta, y que varias discusiones sobre el tema señalan, es de gobierno: *¿quién es responsable de mantener las definiciones una vez escritas, y quién decide cuando dos dominios discrepan sobre el identificador canónico de una entidad?* Una capa semántica acuerda significado el día del lanzamiento; sin responsables, se degrada.

5. Arquitectura de un knowledge graph empresarial y GraphRAG

5.1. El pipeline de construcción

La Figura 6 presenta una arquitectura de referencia: desde fuentes heterogéneas hasta agentes que usan el grafo para responder y actuar.

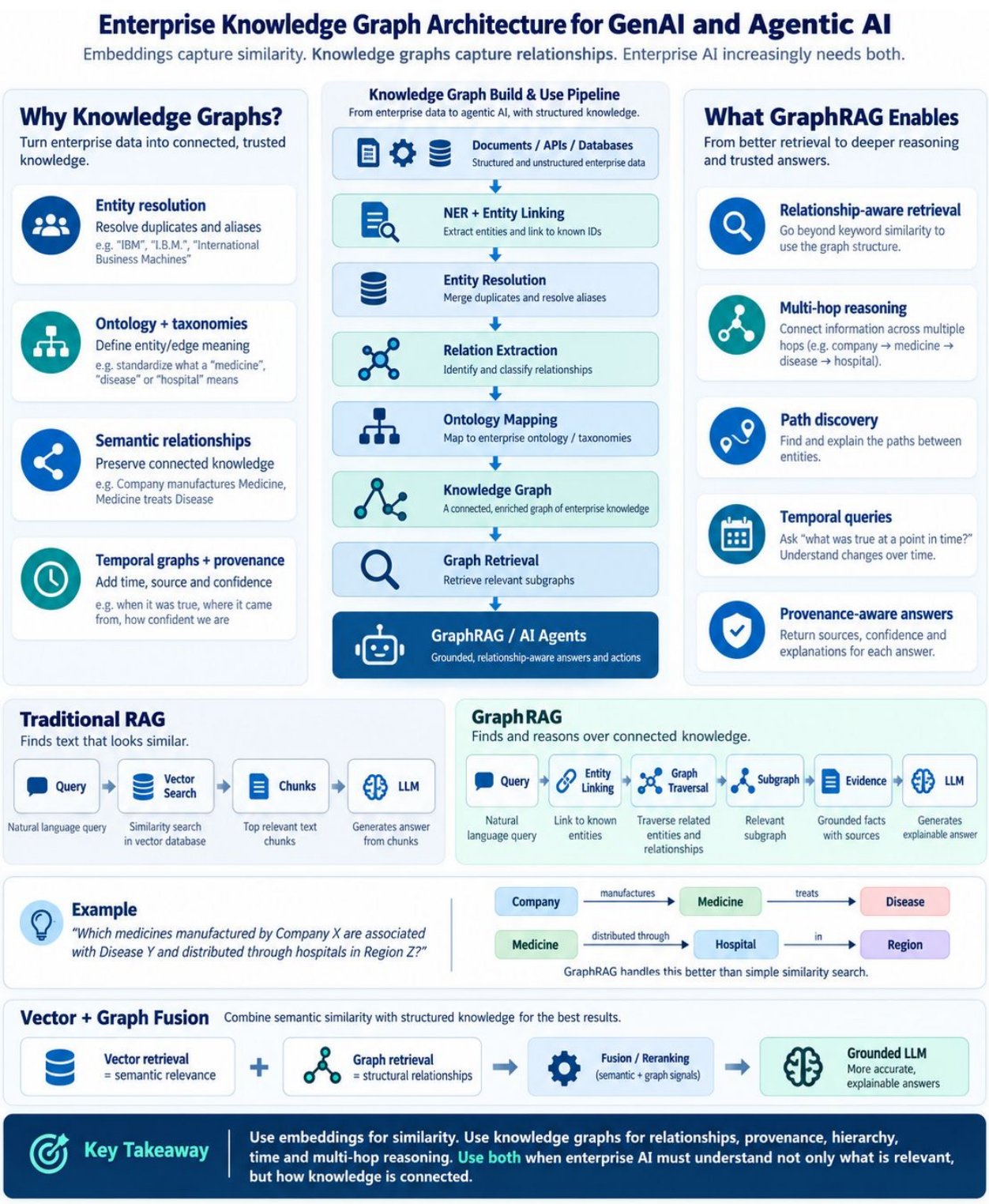


Figura 6. Arquitectura de un knowledge graph empresarial para GenAI e IA agéntica, y comparación entre RAG tradicional y GraphRAG. Fuente: material de divulgación (copia local en fuentes/7503042948104912897/).

El flujo es: documentos, APIs y bases de datos → reconocimiento de entidades nombradas (NER) y enlace de entidades → resolución de entidades → extracción de

relaciones → mapeo a la ontología → knowledge graph → recuperación sobre el grafo → GraphRAG o agentes. Tres decisiones de diseño merecen atención:

1. **La resolución de entidades es previa a las relaciones.** Sin ella, «Microsoft», «Microsoft Corp.» y «MSFT» generan nodos distintos y el grafo queda fragmentado.
2. **La ontología da significado a nodos y aristas.** Un grafo sin modelo semántico es solo una red de objetos; define qué quiere decir *tratado_por* o *fabricado_por*.
3. **Las relaciones pueden llevar contexto:** proveniencia, grado de confianza, fuente, marca temporal y contexto de seguridad (por ejemplo, el inquilino en una arquitectura multi-tenant). Con eso se pasa de preguntar «¿está la empresa A conectada con el fármaco B?» a «¿cuándo, según qué fuente y con qué confianza?».

5.2. De RAG a GraphRAG

En el RAG tradicional el flujo es consulta → búsqueda vectorial → fragmentos (*chunks*) → modelo de lenguaje. GraphRAG lo reemplaza o complementa con consulta → enlace de entidades → recorrido del grafo → subgrafo → evidencia → modelo. Eso habilita la recuperación sensible a relaciones, el razonamiento de múltiples saltos (*multi-hop*), el descubrimiento de caminos, las consultas temporales y las respuestas con proveniencia. El ejemplo de la figura, «¿qué medicamentos fabricados por la empresa X se asocian con la enfermedad Y y se distribuyen en hospitales de la región Z?», ilustra el tipo de pregunta que una búsqueda por similitud difícilmente resuelve.

El material insiste en que el planteo correcto rara vez es «base vectorial *o* grafo»: la recuperación vectorial aporta relevancia semántica y la recuperación sobre el grafo aporta relaciones estructurales; ambas se fusionan y re-ordenan (*fusion / reranking*) antes de entregar contexto al modelo.

5.3. ¿Vale la pena el costo?

GraphRAG no es una mejora gratuita. Un artículo de análisis revisado para este reporte (consultado en línea; no se conserva copia local) distingue cuatro familias de arquitectura:

Familia	Idea central
Jerarquías de comunidades (Microsoft GraphRAG)	Extrae entidades y relaciones, agrupa nodos en comunidades (algoritmo de Leiden) y resume por niveles; admite búsquedas locales y globales
Grafos entidad-relación (LightRAG)	Grafo más plano, sin detección de comunidades; separa palabras clave de bajo nivel (entidades) y de alto nivel (temas)
Recorrido inspirado en memoria (HippoRAG)	Construye tripletas con extracción abierta de información y propaga relevancia con *Personalized PageRank*
Resumen recursivo (RAPTOR)	Agrupa fragmentos similares recursivamente y crea un árbol de abstracciones

Según ese artículo, citando una evaluación controlada, la mayoría de las diferencias relativas de desempeño entre métodos fueron menores al 8 %, mientras que el consumo de tokens por consulta varió mucho: Microsoft GraphRAG habría usado alrededor de 16,6 veces más tokens que LightRAG y 77 veces más que el RAG vectorial estándar. *(Estas cifras provienen de un resumen automático del artículo y no se contrastaron con el estudio original: verificar antes de citarlas.)* La lección operativa es la que el propio material formula: la estructura adicional trae más indexación, más latencia, más tokens y más complejidad, y se justifica cuando la precisión *multi-hop* compensa el costo, no como mejora por defecto. Una idea práctica, sugerida en los comentarios del post y no verificada, es anteponer un enrutador barato que pague el «costo del grafo» solo para las consultas que lo necesitan.

6. Más allá del grafo de conocimiento: de «qué es verdad» a «qué hacer»

Aun con un grafo de conocimiento excelente, un agente puede no saber qué hacer. El material identifica dos extensiones que atacan ese vacío y, más adelante, una arquitectura que las integra.

6.1. Grafo de contexto

La Figura 7 contrasta el grafo de conocimiento (lo que es generalmente cierto del dominio) con un grafo de contexto armado en tiempo de ejecución para una decisión concreta, usando como caso la reubicación de un pasajero en otro vuelo.

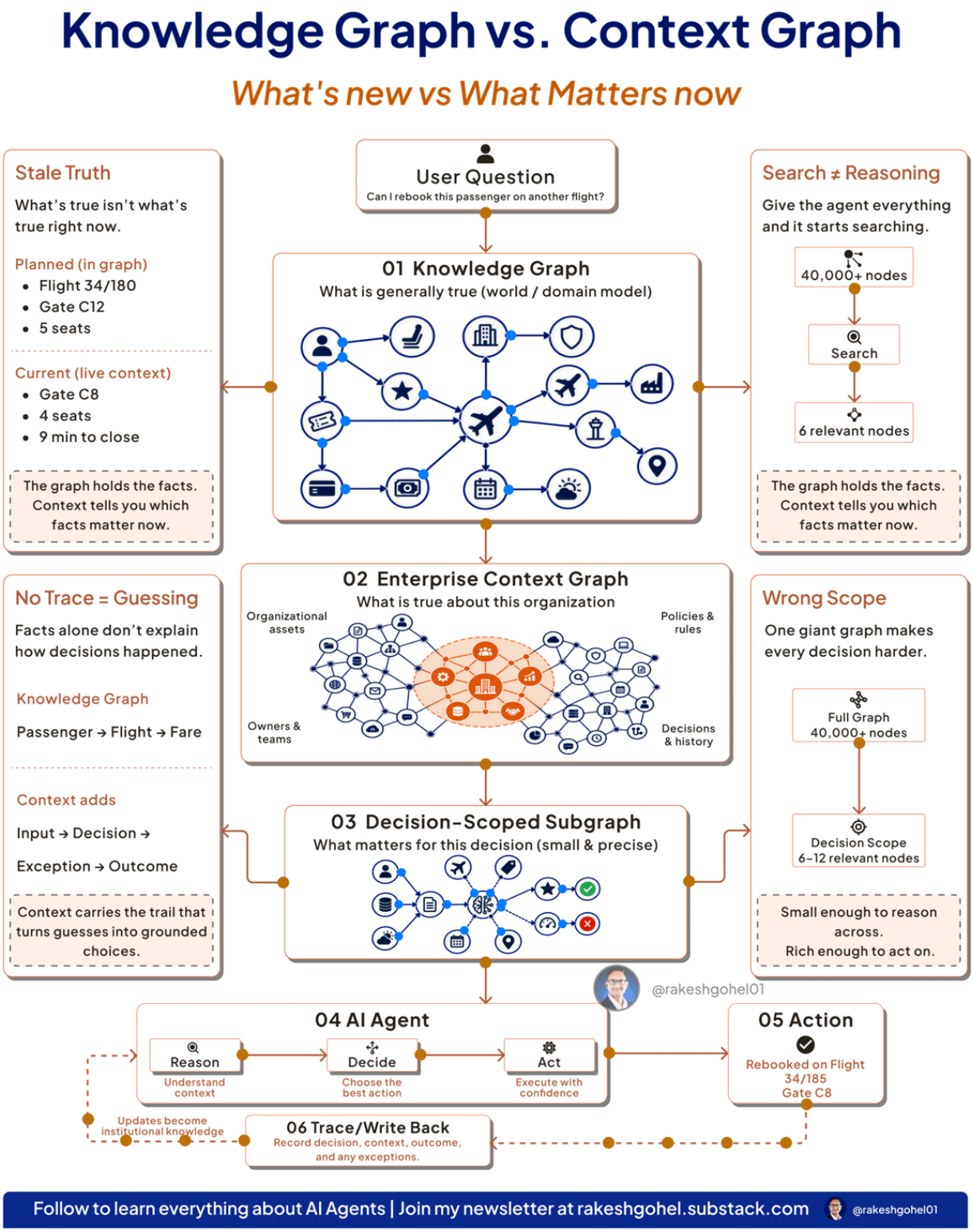


Figura 7. Knowledge graph frente a grafo de contexto: cuatro modos de falla y un flujo de decisión con subgrafo acotado y trazabilidad. Fuente: material de divulgación (copia local en fuentes/ 7495821034198188032/).

Se identifican cuatro modos de falla de un agente que solo cuenta con el grafo de conocimiento:

1. **Verdad obsoleta.** Lo planificado (puerta C12, cinco asientos) difiere de lo actual (puerta C8, cuatro asientos, nueve minutos para el cierre). El grafo contiene hechos; el contexto indica cuáles importan ahora.
2. **Búsqueda no es razonamiento.** Entregar el grafo completo convierte la decisión en un problema de recuperación: puntuar similitud sobre decenas de miles de nodos para quedarse con seis.
3. **Sin traza, adivinanza.** Los hechos no explican cómo se resolvieron casos anteriores; falta el rastro entrada → decisión → excepción → resultado.
4. **Alcance equivocado.** Un grafo gigante para trabajo abierto es difícil de construir y de actuar; acotarlo a la decisión (un subgrafo de 6 a 12 nodos en el ejemplo de la figura) hace el razonamiento tratable.

La propuesta es un flujo en el que el agente razona sobre un *subgrafo acotado a la decisión*, actúa y *escribe de vuelta* la decisión, el contexto y las excepciones, de modo que el conocimiento institucional se actualice. *(El material afirma que el término «grafo de contexto» figura en un informe de analistas del sector; no lo verificamos y no lo usamos como argumento.)*

6.2. Grafo procedimental

La Figura 8 reproduce una figura de un artículo académico que lleva la distinción al extremo: un grafo de conocimiento responde preguntas del tipo *qué-es* con tripletas (entidad, relación, entidad); un **grafo procedimental** responde preguntas *qué-hacer* con tripletas (procedimiento, relación, procedimiento).

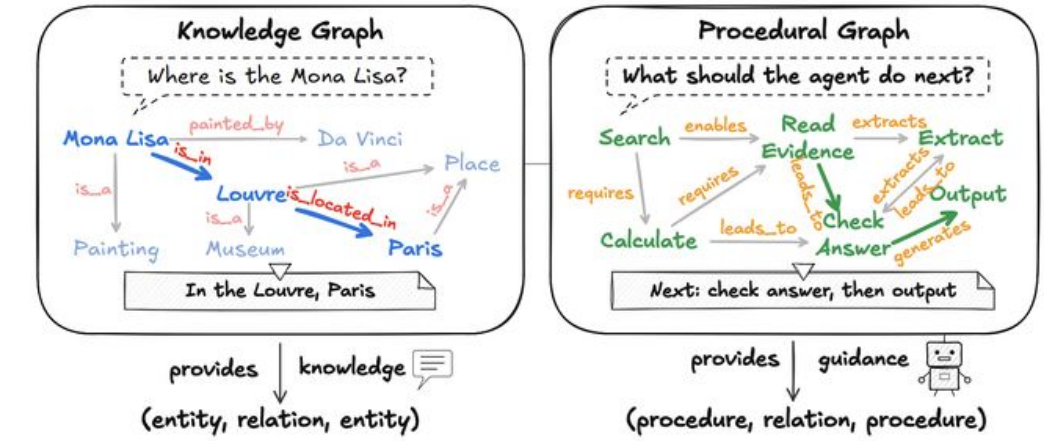


Figure 1 | **From knowledge to procedure.** A Knowledge Graph organizes facts into triplets that answer *what-is* questions. A Procedural Graph organizes task procedures into triplets that answer *what-to-do* questions.

Figura 8. Del conocimiento al procedimiento: grafo de conocimiento (qué-es) frente a grafo procedimental (qué-hacer). Fuente: figura de un artículo académico difundida en redes (copia local en fuentes/7506287140541009920/).

El artículo existe y está publicado en arXiv (*Procedural Graphs: Self-Evolving Execution Structures for LLM Agents*, arXiv 2609.09153): organiza el conocimiento procedimental en tripletas y emplea un refinador basado en un modelo de lenguaje que contrasta trayectorias fallidas con exitosas para editar el grafo. Según la descripción del material, los nodos son llamadas a herramientas, pasos de razonamiento o estados de la tarea, y cada arista lleva tres campos de texto (condición, guía y trampas). En inferencia el grafo está congelado: el sistema empareja la última acción del agente con un nodo, toma su vecindario a dos saltos y genera un consejo situacional que se agrega al *prompt*; sesga la siguiente acción pero no la restringe. Cada modificación propuesta debe igualar o superar el puntaje actual en un conjunto reservado, o se revierte.

Dos cautelas. Primero, los resultados numéricos que cita el material (primer o primer-empatado lugar en 21 de 24 configuraciones, y una mejora puntual en un *benchmark* de llamadas a herramientas) **no pudimos confirmarlos** en la página de resumen del artículo; solo la afirmación general de «mejoras consistentes frente a líneas base de memoria». Segundo, el propio material señala una limitación importante: en una tarea de preguntas multi-documento los márgenes fueron prácticamente nulos, lo que sugiere que la estructura procedimental solo rinde donde el *orden* de las acciones es determinante, y que muchos supuestos «problemas de memoria» son tareas que nunca necesitaron procedimiento. Es una buena regla de prudencia para cualquier decisión de diseño.

6.3. Los tres grafos juntos: la «columna semántica»

Las dos subsecciones anteriores pueden leerse como piezas sueltas. Un material posterior las integra en una sola arquitectura a partir de tres preguntas que un agente debería poder responder: *¿qué es verdad?* (grafo de conocimiento: entidades y reglas estables, gobernadas, de toda la empresa), *¿qué importa ahora?* (grafo de contexto: un subgrafo pequeño con el estado en vivo y el historial de decisiones) y *¿qué hago después?* (grafo procedimental: tripletas procedimiento-relación-procedimiento que *guían* el siguiente paso sin dictarlo). La Figura 9 muestra el conjunto con el caso del vuelo.

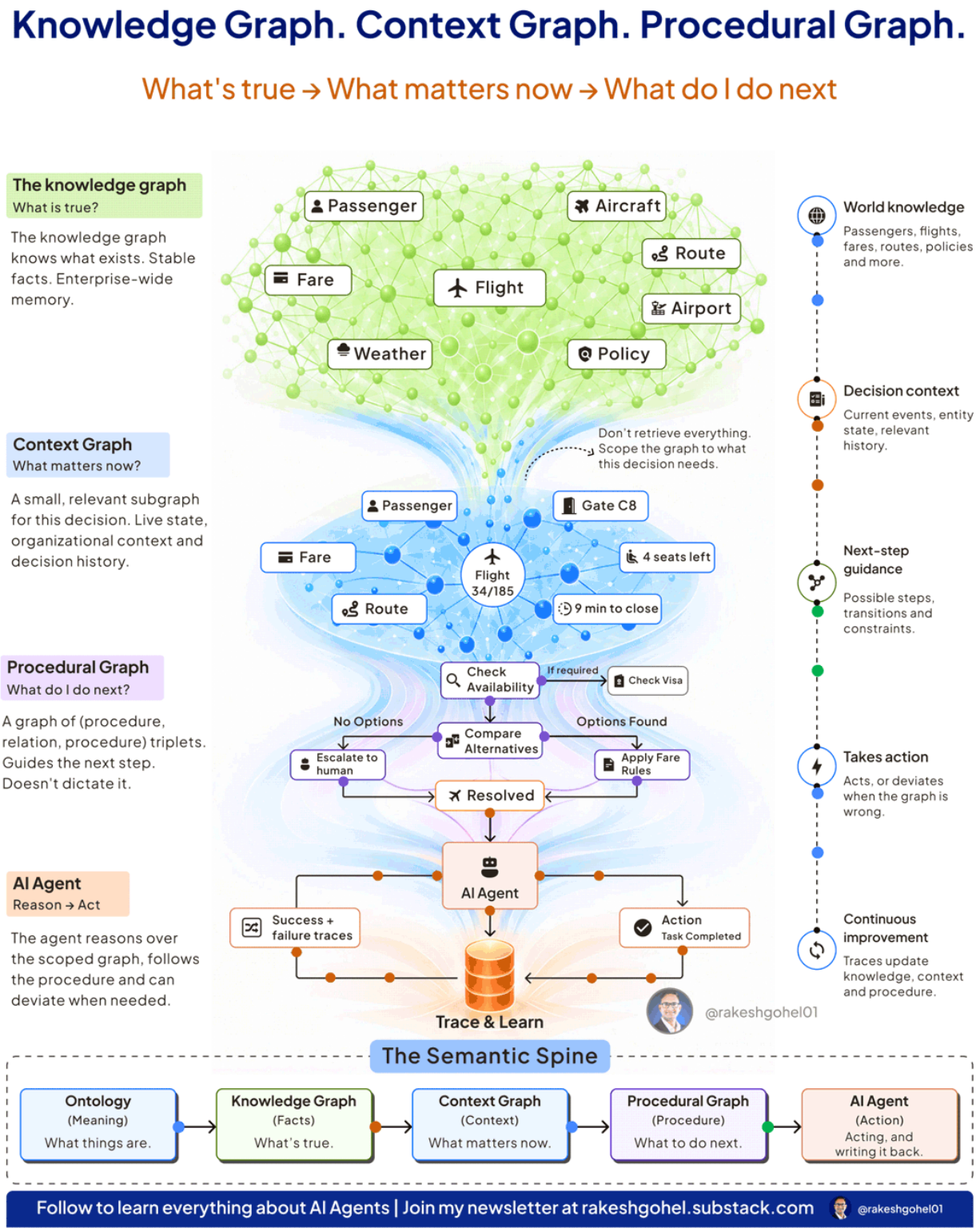


Figura 9. Knowledge graph, grafo de contexto y grafo procedimental integrados en un flujo de decisión, con la «columna semántica» de ontología a acción. Fuente: material de divulgación (copia local en fuentes/7511039116952637440/).

El argumento se apoya en qué falla cuando falta cada pieza:

Si falta…	Qué ocurre
Grafo de conocimiento	El agente no tiene una base confiable sobre la cual apoyarse
Grafo de contexto	Busca por similitud entre decenas de miles de nodos para dar con los seis relevantes: recuperación, no razonamiento
Grafo procedimental	Genera sobre un historial que crece; en tareas largas pierde el objetivo, llama herramientas fuera de orden y entra en bucles

El ciclo en producción ante la petición «reubicar a este pasajero» tiene cuatro pasos: (1) el grafo procedimental indica el paso activo (*verificar disponibilidad*); (2) el grafo de contexto extrae del de conocimiento solo lo que ese paso necesita (este vuelo, cuatro asientos, puerta C8, nueve minutos para el cierre); (3) el agente actúa guiado por la siguiente arista, con libertad para desviarse (por ejemplo, escalar a una persona si no hay opciones, o revisar la visa «si corresponde»); (4) la traza se escribe de vuelta. Cada vuelta mejora los tres grafos: las trazas dan precedente a la próxima decisión, las ejecuciones fallidas corrigen el procedimiento y los patrones repetidos se convierten en hechos gobernados.

De ahí salen tres principios de diseño: acotar el grafo por paso en lugar de entregar el grafo completo, almacenar el procedimiento como grafo y no como un *prompt* largo, y registrar cada decisión como rastro de auditoría. La «columna semántica» ordena las capas en una cadena de responsabilidades únicas: significado (ontología) → hechos (grafo de conocimiento) → contexto (grafo de contexto) → procedimiento (grafo procedimental) → acción (agente), con aprendizaje de vuelta.

Lectura crítica. Es una propuesta de arquitectura conceptual de divulgación: el material no presenta una implementación ni evidencia empírica propia. Dos cautelas conectan con secciones anteriores. Primero, la recomendación de reemplazar *prompts* por un grafo procedimental debe leerse junto con la advertencia de la sección 6.2: la estructura procedimental solo rinde cuando el orden de las acciones es determinante, así que conviene decidir por tarea y no por principio. Segundo, «registrar cada decisión» es necesario pero no suficiente para auditar: como se vio en la sección 8, la traza debe incluir además las versiones de los hechos que el agente leyó, la autoridad bajo la cual actuó y la llamada a herramienta que siguió.

7. Ontologías vivas

Un riesgo recurrente es que una ontología construida a mano sea una *fotografía* desde el primer día: los datos y las cargas de trabajo cambian, las definiciones dejan de coincidir con el trabajo real, nadie tiene tiempo de reescribirla y los agentes vuelven a inferir el significado en cada tarea, pagándolo en tokens. La Figura 10 resume una propuesta alternativa: una ontología que el agente «entrena» mientras trabaja.

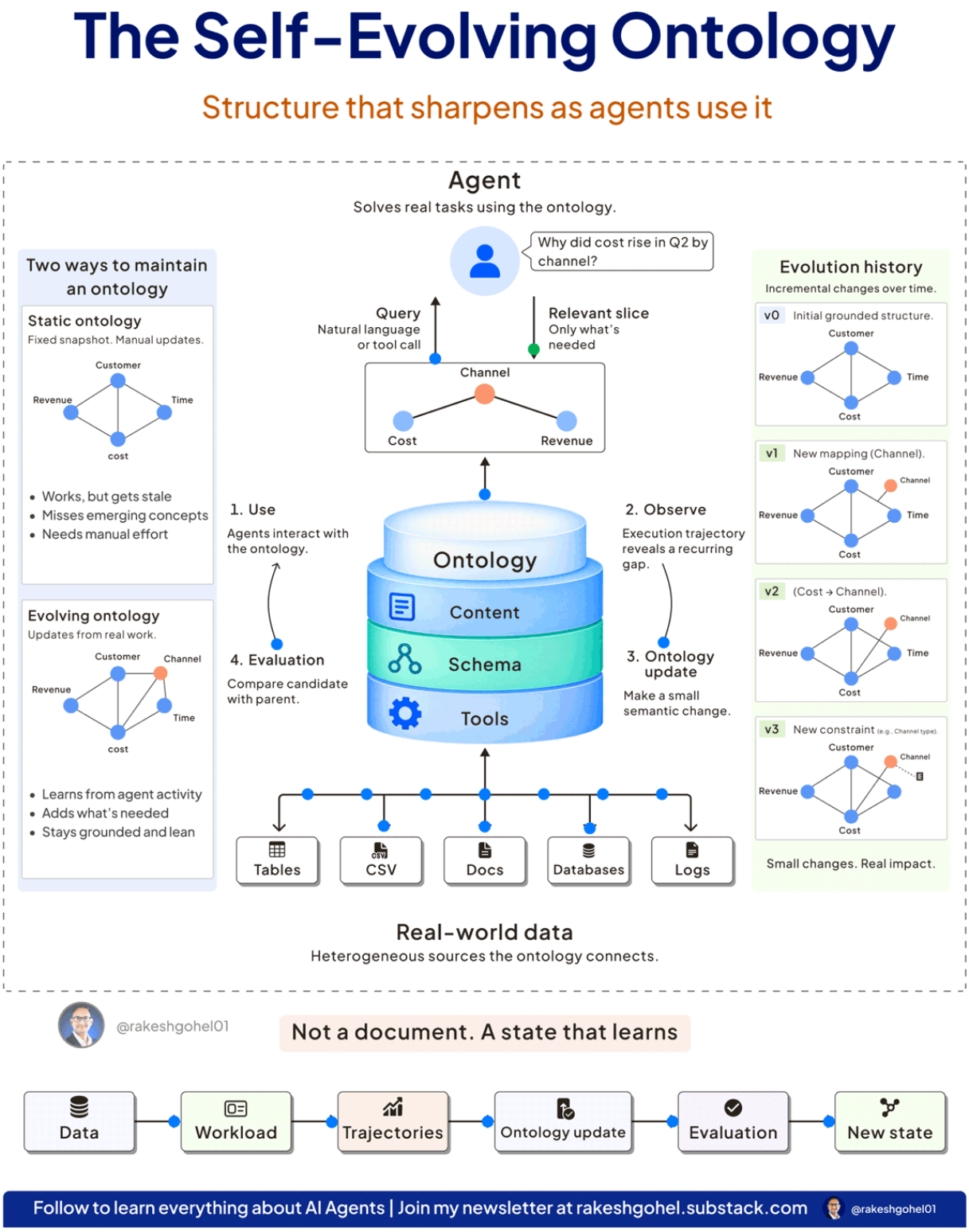


Figura 10. Ontología autoevolutiva: ciclo uso → observación → actualización → evaluación, con historial de versiones. Fuente: material de divulgación (copia local en fuentes/7507777609552875520/).

El trabajo académico detrás de la figura (*EvoOntology: A Self-Evolving Ontology Layer for Data Agents*, arXiv 2609.15779, septiembre de 2026) describe una capa ontológica expuesta como servidor MCP con tres componentes (esquema, contenido y herramientas), un agente constructor que la genera de forma autónoma y un ciclo de

autorrefinamiento con «ediciones tipadas guiadas por atribución» que solo se aceptan tras una evaluación pareada. Los autores informan mejoras consistentes en tres *benchmarks* de agentes de datos con cuatro modelos de lenguaje. Esos tres rasgos coinciden con lo que enfatiza la figura: aprende del trabajo real, cambia solo si el cambio gana frente a la versión anterior con igual tarea y presupuesto (y cada cambio es inspeccionable y reversible), y crece sin inflar el *prompt*, porque cada paso extrae solo la porción de significado que necesita.

Cuestión para el debate. ¿Confiaría en que un agente evolucione por sí solo la ontología de su organización? Una respuesta razonable es «ni totalmente sí ni totalmente no»: nadie puede mantenerla a mano, pero tampoco conviene dejarla correr sin supervisión. Esto conecta con la advertencia de la sección 4 sobre responsables de las definiciones, y con la necesidad de un especialista (un ontólogo) en el circuito.

8. Gobernanza y auditoría: proveniencia no es auditoría

Es tentador asumir que, si el grafo registra el origen de cada hecho, el sistema de IA es auditable. Una de las observaciones más agudas del material lo discute: la **ingeniería del conocimiento hace *explicable la recuperación*, lo cual es un alcance más estrecho que la auditabilidad**. Las columnas de proveniencia y linaje (ver Figura 4) responden de dónde proviene un hecho. No registran qué *versión* leyó el agente, bajo qué *autoridad delegada* actuó ni qué llamada a herramienta siguió. Cuando los agentes escriben de vuelta en los mismos sistemas, el grafo deja de ser un registro de una decisión para convertirse en una *entrada* a la decisión.

Una prueba interna útil, propuesta en el material: tomar una acción completada por un agente y tratar de reconstruir los hechos exactos, las versiones y los permisos vigentes en ese momento. Si el linaje explica el documento pero no la acción, la capa de evidencia está incompleta. La conclusión: la proveniencia debe estar en la ruta de ejecución y en evidencia retenida, no solo en el modelo de conocimiento.

Una línea complementaria, tomada de una charla recomendada en el material (que no pudimos revisar para este reporte; se la menciona como lectura sugerida, no como fuente), sostiene que las ontologías pueden funcionar como *barandas de seguridad* de dominio para envolver a un modelo probabilístico en lógica del negocio, por ejemplo para impedir reembolsos duplicados sobre un mismo pedido o estados inventados como «probablemente enviado». Es una idea consistente con el uso de restricciones y reglas en una ontología (Figura 3) y merece ser contrastada con la charla original.

9. Nota de desambiguación: «grafo» en *graph engineering*

Un cuidado terminológico final. En la literatura sobre agentes aparecen también las expresiones *loop engineering* y *graph engineering*, y la palabra «grafo» allí **no** significa knowledge graph. La Figura 11 lo aclara.

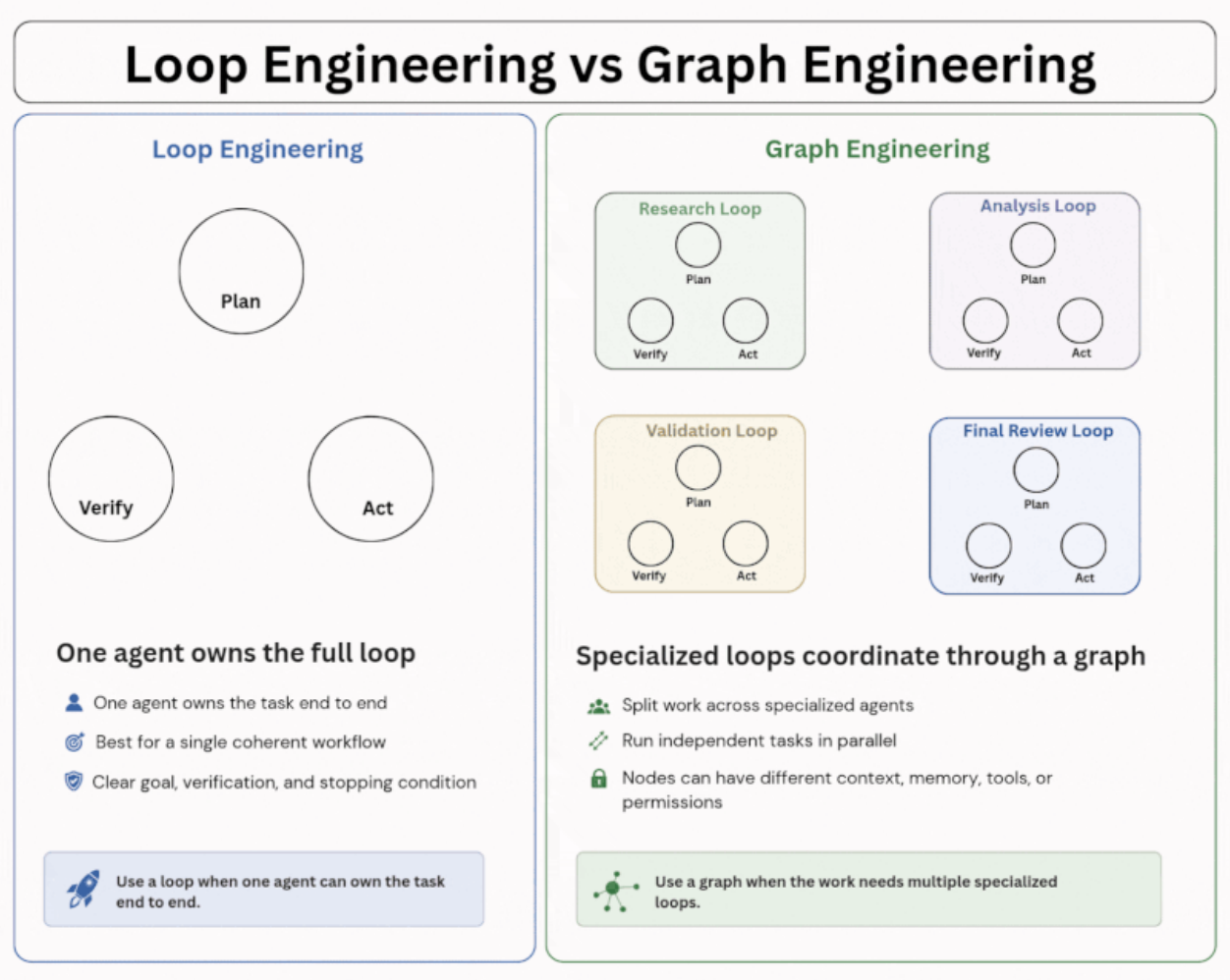


Figura 11. Loop engineering frente a graph engineering en la orquestación de agentes. Fuente: material de divulgación (copia local en fuentes/7503818218700984322/).

Un **bucle** es un ciclo autónomo *planificar → actuar → verificar → repetir* hasta una condición de parada; la ingeniería de bucles busca hacerlo confiable (qué contexto sobrevive entre iteraciones, cómo se autoevalúa el agente, cuándo reintenta, cómo termina). Un **grafo**, en este sentido, es un *grafo de flujo de trabajo*: los nodos son unidades de trabajo (agentes, herramientas, validadores, funciones deterministas) y las aristas definen cómo se mueven el estado y el control. No compiten: un bucle puede ser un nodo dentro de un grafo mayor. Se empieza con un bucle cuando un agente puede resolver la tarea de punta a punta, y se pasa a un grafo cuando se necesita ejecución en paralelo, agentes especializados, permisos distintos o ramificación explícita. Es un grafo de *control*, mientras que los de las secciones anteriores son grafos de *conocimiento* y de *contexto*; conviene no mezclarlos.

10. Síntesis y criterios de decisión

Tabla 2. ¿Qué usar y cuándo? (elaboración propia a partir del material).

Situación	Herramienta indicada
Etiquetado y navegación consistentes (categorías de productos)	Taxonomía
Entendimiento compartido de un dominio y consultas con razonamiento	Ontología
Integrar o migrar datos entre sistemas con vocabularios distintos	Mapeo semántico
Consultas en lenguaje natural sobre datos estructurados con métricas consistentes	Modelo / capa semántica
Preguntas sobre relaciones entre entidades, multi-hop, proveniencia y tiempo	Knowledge graph + GraphRAG
Agente que debe decidir con información vigente	Subgrafo de contexto acotado a la decisión
Tareas donde importa el orden de las acciones	Grafo procedimental (evaluar si realmente hace falta)
Orquestar varios agentes con permisos y estados distintos	Grafo de flujo de trabajo (*graph engineering*)

Ideas que conviene retener.

1. *Significado antes que relaciones:* sin definiciones acordadas y gobernadas, un grafo es otra capa de datos desordenada.
2. *Vectores y grafos se complementan:* similitud para relevancia, estructura para relaciones, jerarquía, tiempo y razonamiento *multi-hop*.
3. *Un grafo de conocimiento dice qué es verdad, no qué hacer ni qué importa ahora:* para eso se necesita contexto acotado a la decisión.
4. *Más estructura cuesta más:* más indexación, latencia y tokens; hay que justificarla con el tipo de pregunta.
5. *La proveniencia explica la recuperación, no la acción:* la auditoría exige registrar versiones, permisos y llamadas a herramientas en la ejecución.
6. *La semántica no es estática:* una ontología debe poder evolucionar, con cambios evaluados, reversibles y supervisados.

Preguntas para discusión.

1. Tome un dominio que conozca (banca, salud, logística) y proponga dos clases, tres relaciones y una restricción para su ontología. ¿Cuáles de esos elementos pertenecerían a la capa semántica y cuáles a la ontología?

2. Dada una consulta de su dominio, ¿requiere realmente *multi-hop*? Estime qué ganaría y qué perdería (tokens, latencia, mantenimiento) al pasar de RAG vectorial a GraphRAG.
3. Diseñe la prueba de reconstrucción de la sección 8 para un agente concreto: ¿qué registraría en cada acción para poder reproducir hechos, versiones y permisos?
4. ¿En qué tipos de tarea esperaría que un grafo procedimental no aporte nada? Justifique con la sección 6.2.

Material de origen

Sección	Material	Copia local
2	Infografías sobre modelo semántico, ontología y grafo; y sobre taxonomía, ontología y mapeo semántico	`fuentes/7508657406294298624/`, `fuentes/7478312221530312705/`
3	Infografías de seis conceptos de IA empresarial y glosario de 22 términos	`fuentes/7511047409553199104/`, `fuentes/7507426768228679680/`
4	Infografía sobre la capa semántica como cimiento	`fuentes/7508064074430181376/`
5.1–5.2	Infografía de arquitectura de knowledge graph empresarial y GraphRAG	`fuentes/7503042948104912897/`
5.3	Post sobre GraphRAG y artículo de análisis asociado (consultado en línea)	`fuentes/7508553973751226368/` (solo texto)
6.1	Infografía sobre knowledge graph y grafo de contexto	`fuentes/7495821034198188032/`
6.2	Post y figura sobre el grafo procedimental; artículo arXiv 2609.09153	`fuentes/7506287140541009920/`
6.3	Infografía sobre los tres grafos integrados y la «columna semántica»	`fuentes/7511039116952637440/`
7	Infografía sobre ontología autoevolutiva; artículo arXiv 2609.15779	`fuentes/7507777609552875520/`
8	Post sobre ingeniería del conocimiento y auditabilidad; post sobre una charla (sin revisar la charla)	`fuentes/7507426768228679680/`, `fuentes/7501355891091415040/` (solo texto)
9	Infografía sobre loop y graph engineering	`fuentes/7503818218700984322/`

Notas sobre el material.

- Las infografías son material de terceros y conservan la marca de agua o el nombre de su autor incrustados en la imagen, que no se pueden retirar. Si el reporte se va a publicar, conviene redibujarlas (por ejemplo, con Mermaid o draw.io) o solicitar autorización.
- Quedaron fuera de este reporte, por no haberse leído todavía, dos videos sobre *graph engineering*, un repositorio y su sitio sobre *loop engineering*, y los comentarios de los posts no se usaron como fuente.
- **Redactado con conocimiento propio (no proviene del material):** la precisión sobre RDF/RDFS/OWL y la distinción esquema/instancias de la sección 2, la tabla 2 y las ideas de cierre.
- **Pendiente de contrastar con documentación oficial antes de la versión final:** las cifras de costo en tokens de la sección 5.3 (resumen automático de un artículo), los resultados numéricos del grafo procedimental (sección 6.2), las descripciones de arquitecturas (Microsoft GraphRAG, LightRAG, HippoRAG, RAPTOR) contra sus papers y repositorios, y los estándares de la web semántica contra la documentación de W3C. Las afirmaciones del material sobre informes de analistas y adopción del mercado no se verificaron y no se usan como argumento.